FACULTAD DE INGENIERÍA, ARQUITECTURA Y DISEÑO

ESTUDIANTES/CÉDULA:

CARLOS POVEDA // 8-976-1952

BASE DE DATOS II

ACTIVIDAD:

PARCIAL 2

PROFESOR(A):

MARYON JOSE TORRES R.

Informe Técnico: Sistema de Gestión Jurídica

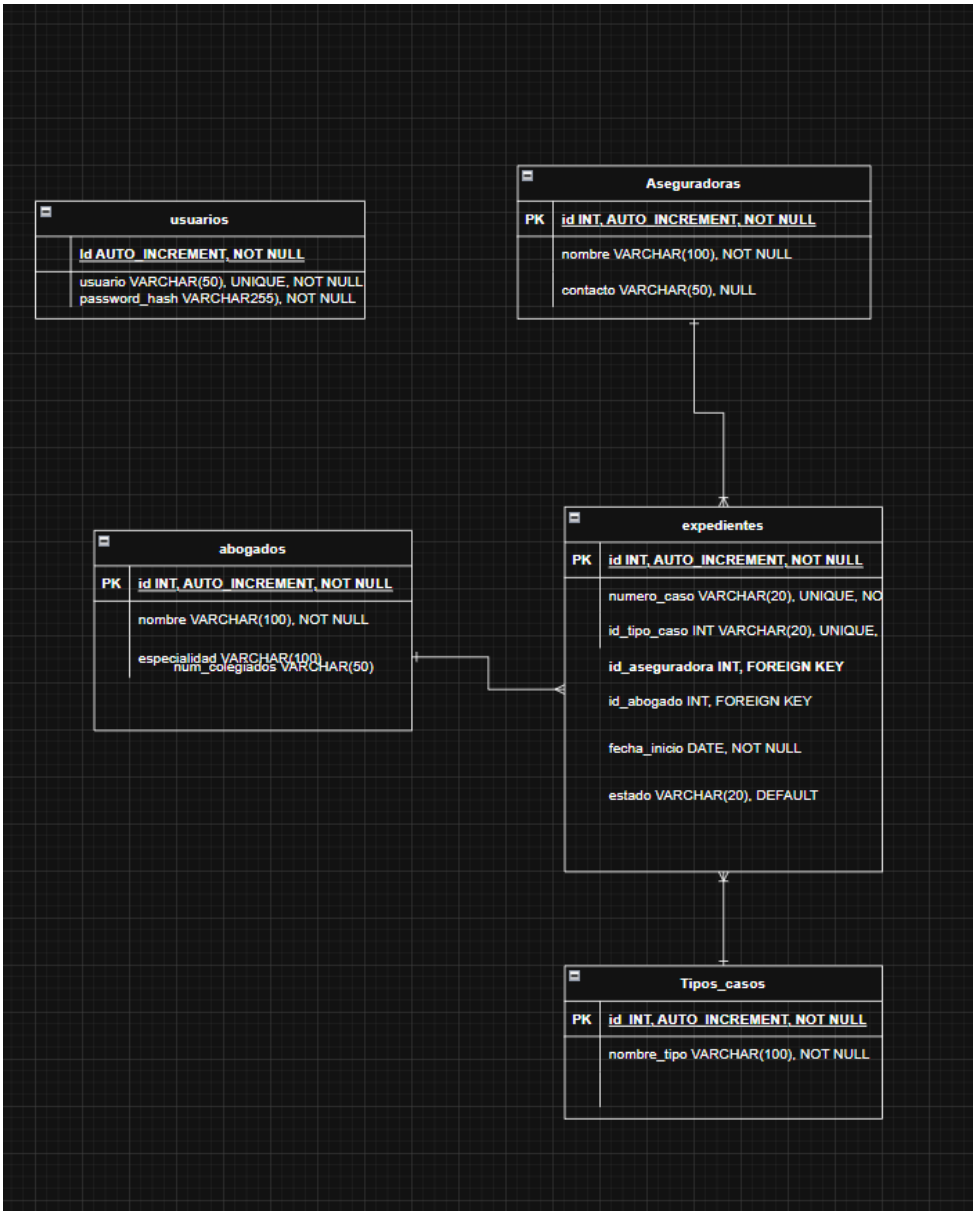
Proyecto: Prototipo de API REST para Control de Expedientes y Aseguradoras.

1. Diseño de la Base de Datos

El sistema se diseñó bajo un modelo relacional para gestionar la persistencia de datos de un bufete de abogados, conectando los expedientes con sus respectivos actores (aseguradoras, abogados y catálogos).

Diagrama Entidad-Relación (ERD)

El modelo cuenta con 5 tablas principales. La tabla expedientes actúa como el nodo central que relaciona las llaves foráneas de las demás entidades.



2. Normalización (3FN)

Para asegurar la integridad y evitar la redundancia, se aplicaron las tres primeras formas normales:

- **1FN (Primera Forma Normal):** Se definieron llaves primarias únicas para cada tabla y se aseguró que cada campo contenga valores atómicos (sin listas de teléfonos o nombres múltiples en una sola celda).
- **2FN (Segunda Forma Normal):** Se eliminaron las dependencias parciales. Atributos como el nombre de la aseguradora o la especialidad del abogado se movieron a sus propias tablas, ya que no dependen de la clave del expediente, sino de su propio ID.
- **3FN (Tercera Forma Normal):** Se eliminaron las dependencias transitivas. Ninguna columna no clave depende de otra columna no clave; toda la información se accede mediante relaciones directas de ID.

3. Implementación SQL (MariaDB)

Se utilizó **MariaDB** ejecutándose en el puerto **3308:3306**. Se implementaron 3 vistas para simplificar el consumo de datos desde la API:

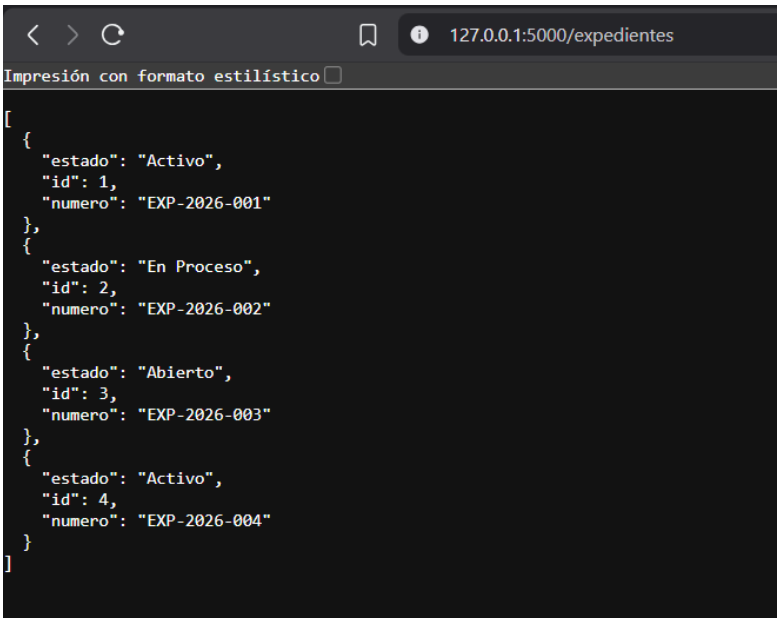
- **vista_detalle_expedientes:** Une las tablas para mostrar nombres en lugar de IDs.
- **vista_estadistica_aseguradoras:** Realiza un conteo (COUNT) de casos por cada empresa de seguros.
- **vista_carga_abogados:** Muestra cuántos casos activos tiene asignado cada licenciado.

4. Integración y API REST (Python + Flask)

Se desarrolló un **Producto Mínimo Viable (MVP)** utilizando el micro-framework Flask y SQLAlchemy como ORM para la conexión al puerto 3308.

Funcionalidades Implementadas:

- **Conector Relacional:** Uso de pymysql para la comunicación fluida con MariaDB.
- **GET /expedientes:** Endpoint para consultar la lista completa de registros en formato JSON.
- **POST /expedientes:** Endpoint para la creación de nuevos registros mediante el envío de objetos JSON.



5. Lecciones Aprendidas

1. **Gestión de Puertos:** Aprendí la importancia de configurar correctamente el mapeo de puertos (3308) cuando existen múltiples instancias de bases de datos para evitar colisiones.
2. **Abstracción SQL:** El uso de **Vistas** facilita enormemente el trabajo del programador de Backend, ya que la base de datos entrega la información ya procesada.
3. **Seguridad de Datos:** La implementación de un módulo de Login requiere pensar en la integridad de la tabla de usuarios antes de exponer cualquier dato sensible.
4. **Estandarización JSON:** La API REST permite que diferentes sistemas (móviles o web) consuman la misma información de manera estándar y ligera.
5. **ORM vs SQL Puro:** El uso de SQLAlchemy permite escribir código más limpio y legible en Python, delegando la complejidad de las consultas al motor de base de datos.

6. Guía de Ejecución (Para Revisión)

1. **Base de Datos:** Importar el script database_script.sql en el puerto 3308.
2. **Librerías:** Instalar dependencias con pip install flask flask-sqlalchemy pymysql.
3. **Servidor:** Ejecutar python app.py.
4. **Prueba:** Acceder a http://127.0.0.1:5000/expedientes para validar el JSON.